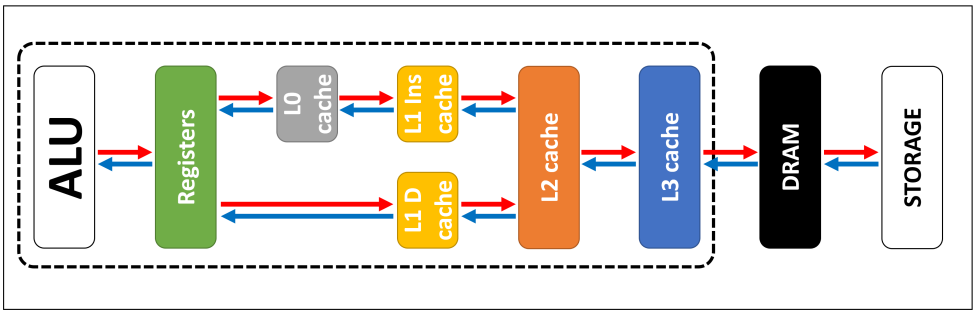
1 Lesson 30-10-26

1.0.1 Cache levels

We talked for a while about cache memories, but not about how caches are actually organized on the local bus.

We find that they are usually close to the *processors*, organized in **levels** progressively farther from it. Usually, at level 1 only, we have a split data and instruction cache.



t may be convenient to have multiple levels of caches, as we can have:

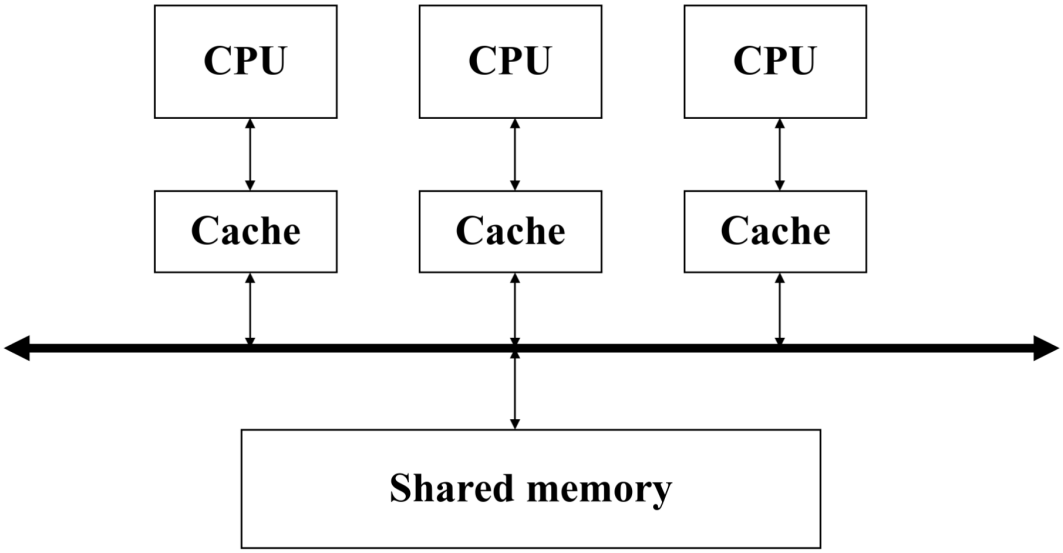
- A first level cache (L1), smaller and faster;
- A second level cache (L2), slower but larger;
- A third level cache (L3), even slower and bigger.

Each time the processor performs a memory access

1. It checks first whether the word is in L1
2. If so, it can access the word in L1
3. If not, it checks whether the word is in L2
4. If so, it will access to the word in L2 and eventually update L1
5. If not, it checks whether the word is in L3
6. If yes, L3 is accessed and L2 is updated
7. If not, it accesses to the main memory and eventually updates L3

1.0.2 Cache coherence

For multiple processor CPUs, then, we are intersted in *cache coherence* between multiple processor caches. The architeture is usually as follows:



The problem is that of course we don't know if all CPUs get the same value in their cache: even if we implement a *write through* (see 12.0.4) policy, the other caches on the bus don't notice the value change if they don't explicitly go and check if they have the updated value in their cachelines.

This is a major problem in multiprocessor systems with shared memory, in which each processor has its own cache. Similar problems may occur if the system uses a DMA controller.

To fix this and achieve cache coherence, two main approaches exist:

- **Snooping** Snooping is a process where the individual caches monitor address lines for accesses to memory locations that they have cached. If a write is detected in one of the locations contained in cacheline, the cacheline on the cache is invalidated;
- **Directory-based** In a directory-based system, the data being shared is placed in a common directory that maintains the coherence between caches. The directory acts as a filter through which the processor must ask permission to load an entry from the primary memory to its cache. When an entry is changed, the directory either updates or invalidates the other caches with that entry. The directory is usually stored in shared memory.

1.1 Branch prediction

We have somewhat hinted at **branch prediction** in the past lessons, sticking to simple predict taken or predict untaken solutions for pipelined processors. Since we have studied the Tomasulo architecture and its application insSuperscalar processors with ROBs that need flushing on misses (which is a very expensive operation), we know want to focus on more advanced techniques for predicting wether a branch is taken.

To summarize, branches can potentially impact in a very serious way the pipeline performance. It is possible to reduce the performance losses by predicting how branches will behave.

Branche preduction *schemes* aim at correctly forecasting branches, thus reducing the chance that control dependences cause stalls. They can be categorized in two groups:

- **Dynamic techniques**: they are implemented by the hardware based on the behavior of the code;
- **Static techniques**: they are handled by the compiler resorting to a preliminary analysis of the code.

This is akin to the distinction we made in 6.2 regarding ILP dynamic scheduling techniques.

1.1.1 Static branch prediction

Of course, as we said earlier, **static** branch prediction can be useful when combined with other static techniques, such as:

- Enabling delayed branches;
- Rescheduling to avoid data hazards.

The compiler may predict branch behavior in different alternative ways:

- Always predicting branches as taken: this is the simple *predict taken* scheme.

  It is generally inaccurate, with benchmarks giving anything from 5% to 60% misprediction rate;

- Predicting branch behavior depending on branch direction, which is a simple but highly accurate techniques for code in standard form (such as code produced by C compilers).

  The technique is based on the observation that, mainly due to loop constructs in high level languages:

  - Forward branches are more often untaken: the only real cases in high level code is the `goto` statement sometimes used in kernel-level C code to simulate exception handling;
  - Backward branches are more often taken: these are the common branches inside `while` and `for` loops. If a loop repeats n times, then $n - 1$ times the branch is taken, thus the heuristic.

- Predicting on the basis of **profile** information coming from earlier runs. Note that this is *not* dynamic yet (!). We are making heuristic decisions on profile data collected at compile time, which then gets hardcoded.

  So, the technique is based on:

  1. Identifying a typical sequence of input stimula for the program;
  2. Running the program a limited number of times using these stimula;
  3. Gathering statistics on the behavior of branches;
  4. Using these statistics as predictions.

1.1.2 Dynamic branch prediction

Dynamic schemas are based on hardware, and use the branch instruction address to activate the different prediction mechanisms. Dynamic branch prediction can be based on different techniques:

- *Branch History Table* (**BHT**): one- and two-bit prediction schemes.

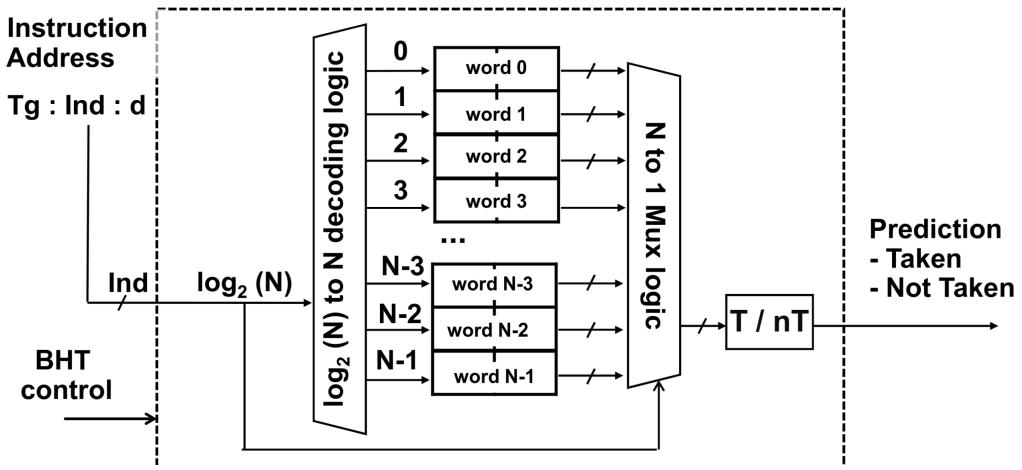
  It is the simplest method for dynamic branch prediction. The BHT is a small memory:

  – Indexed by the lower portion of the address of the branch instruction (same as in cache direct mapping);
  – Containing for each entry one or more bits recording whether the branch has been taken or not the last time it has been executed.

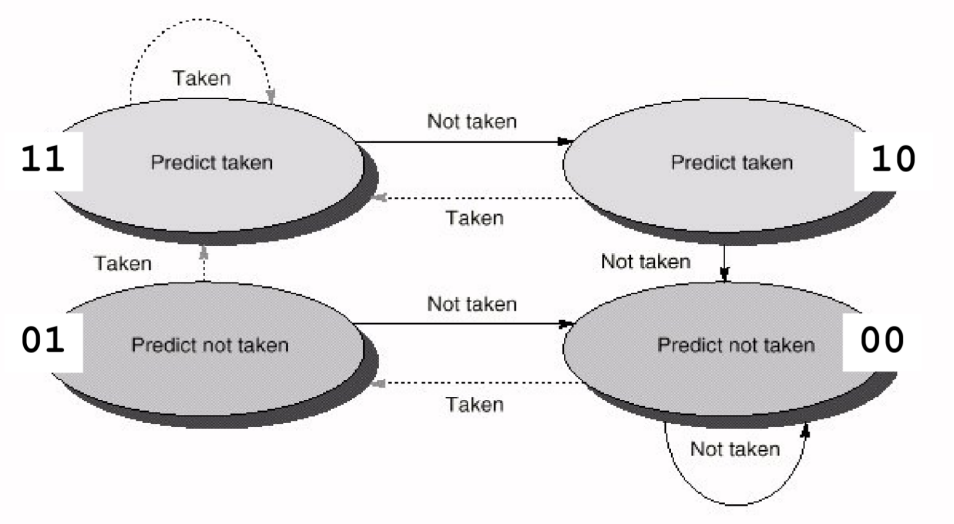
  Each time a branch instruction is decoded:

  1. An access is made in the BHT using the lowest portion of its address as an index
  2. The prediction stored in the table is used, and the new PC is computed according to the prediction.
  3. When the branch result is known, the BHT is possibly updated.

  A BHT using only **one bit** at every table location is called a *Bimodal* predictor



Two bit BHT entries can also be used. They provide higher prediction capabilities. If for every branch, two bits are maintained, the prediction is changed only after missing twice.



- Two-level prediction schemes (correlating predictors);
- *Branch Target Buffer* (**BTB**);
- Others.